SpeakSpace Local

A Local-First Voice Notes and Knowledge Workbench

Individual Project Report

Fan

A dissertation submitted to the University of Bristol in accordance with the requirements for award of the degree of MSc Computer Science in the Faculty of Science and Engineering.

School of Computer Science

August 2026

Word count: 2700

Abstract

SpeakSpace Local is a privacy-oriented desktop workbench that turns recordings into structured notes and makes those notes searchable and usable by local AI. This report reflects on my individual contribution to the group project. My primary responsibilities were selecting and integrating local STT, LLM, embedding and TTS models; designing a bounded Agent workflow with hybrid keyword-vector retrieval; and improving the UI and human-computer interaction. I assisted, rather than led, the system architecture and database design.

The principal technical contribution was an evidence-bounded Agent for a small local language model. It combines explicit note attachment, parallel keyword and semantic retrieval, Reciprocal Rank Fusion, structured tool calls, and a six-step controller. Tool observations are returned to the model, while code enforces scope, cancellation, duplicate-call detection and termination. This approach was motivated by observed failures of prompt-only control. My design work also consolidated recording, review and AI conversation into a single Conversation Studio, with progressive disclosure, direct manipulation, responsive layouts and recoverable status feedback.

Evaluation used isolated retrieval, a 90-task Agent corpus, five-model comparison, 56 human-recorded STT clips, a 36-text TTS benchmark and automated interaction tests. Direct retrieval reached 98.6% Recall@8, but Agent-mediated retrieval only 41.7%; holdout strict completion was 40.0% despite 94.8% fact coverage, locating the main weakness in orchestration. MOSS was fastest but reached 10.4 GiB peak memory on the longest text, supporting MeloTTS as the balanced default. The shared gate recorded 560 passing tests and no failures. A four-machine benchmark confirmed local feasibility but exposed host-sensitive throughput and memory, including a 5.5–18.6 GiB MOSS peak-RSS range. Formal user studies, multi-speaker validation, version-aligned repeat runs and migration testing remain open; engineering proxies are not proof of user experience.

Keywords: local AI; Agent orchestration; hybrid retrieval; speech interfaces

Statement on the inclusion of published and collaborative work

My dissertation does not include work that has been published, submitted or prepared for publication with me as an author.

My dissertation includes non-published collaborative work that contains data or contributions from others. My primary responsibility was local model selection and integration, the evidence-bounded Agent workflow, hybrid keyword-vector retrieval, and UI/HCI optimisation. I assisted with system architecture and database design; these were team decisions and are not presented as my independent work. Shared project test results are identified as such, while my design decisions, implementations, evaluations and reflections are described in the first person.

Author's Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

Fan

31 August 2026

Contents

List of Tables

List of Figures

1. Introduction and Context

Voice notes are easy to capture but difficult to reuse. A recording may contain decisions, deadlines and related ideas, yet remains opaque until it is transcribed, reviewed and organised. Cloud AI reduces this work but requires potentially private meetings, lectures or reflections to leave the device. Local-first research instead emphasises offline operation, data ownership and user control (Kleppmann et al., 2019). SpeakSpace Local applies those principles to a desktop workflow in which recordings, transcripts, structured notes, vectors and AI conversations remain on the user's computer.

The intended beneficiaries are students revising lectures, researchers organising interviews, professionals reviewing meetings and privacy-conscious note takers. They need more than transcription: correction, retrieval, follow-up questions and action extraction must form one usable path. Multilingual speech recognition such as Whisper provides a foundation for local STT (Radford et al., 2023); retrieval- augmented generation motivates explicit external memory rather than relying only on model parameters (Lewis et al., 2020). The project combines these ideas in an Electron application with recording, local models, SQLite persistence, semantic search, Agent tools, speech playback and export.

Local execution also creates the central challenge. Users have different processors, memory budgets and languages; a small model is accessible but less reliable, while a larger model increases download, latency and energy costs. An Agent can retrieve useful evidence but can also search beyond the requested scope or repeat an expensive call. The project therefore treats model choice, privacy, recoverability and control as functional requirements.

My primary responsibility was local STT, LLM, embedding and TTS selection and integration; the Agent workflow and hybrid retrieval; and UI/HCI optimisation. I assisted with architecture, migration and Repository discussions, which were shared team decisions. My goal was a visible path in which models are replaceable, Agent actions are evidence-bounded, and users can understand readiness, context and progress. The result is a functional engineering prototype with

potential to lower the barrier to privacy-preserving knowledge work, but it is not yet proof of equal performance on all hardware or of usability for every target group.

> **Contribution boundary.** Model integration, Agent orchestration, hybrid retrieval and UI/HCI were my primary areas. Architecture and database design were collaborative; I integrated my modules through the team's agreed boundaries.

2. Innovation and Creativity

The main creative design decision was to replace separate transcription and AI pages with one Conversation Studio. The early interface required recording in one place, review elsewhere and a second "current note/workspace" scope choice before asking a question. I treated this fragmentation as an interaction problem rather than adding navigation.

I iterated in a high-fidelity React prototype because recording state, delayed model output, drag targets and narrow-screen behaviour could not be judged from static wireframes alone. Rendered screenshots, computed-style measurements and repeatable interaction paths were documented in dated FAN logs. The final studio combines the note library, conversation, current-note detail, recording, upload and Agent input. Notes can be attached through a hash menu or dragged into the conversation; that visible object is also the exact Agent scope. This links direct manipulation (Shneiderman, 1983) with human-AI guidance to expose context, support correction and provide controls when AI is wrong (Amershi et al., 2019).

Prototyping changed the recording flow. Originally, stopping waited for transcription and semantic processing before opening review, so a working button appeared broken. I opened the dialog immediately in a processing state, then filled the transcript and structured draft as they completed. Destructive actions remain disabled until safe. I later removed a duplicate summary-generation path and reused the structured draft, preventing inconsistent versions of the same note.

The same principle shaped model readiness and onboarding. Instead of silently disabling

speech or Agent actions, a gate names the missing model/runtime and offers recovery. A six-step text tutorial evolved into a 27-step spotlight tour that targets real controls, supports keyboard navigation and can be exited. Accessibility was not separated from visual polish: icon buttons retain labels, separators support keyboard resizing, motion respects reduced-motion preferences, and narrow layouts allocate space rather than overlap panels.

One experiment failed. Frame-by-frame blur made transitions look richer but caused visible stutter on the model page. I removed blur and retained transform and opacity. This established a stronger design rule: novelty had to survive performance and interaction checks in the complete application. The innovation was therefore the alignment of visible context, Agent permission and recoverable feedback, not an isolated visual effect.

3. Technical Development

3.1. Problem and retrieval design

My detailed technical case is the evidence-bounded Agent and hybrid note retrieval. ReAct interleaves model decisions with actions that obtain external information (Yao et al., 2023); my implementation uses this observable action-observation loop without requesting or exposing a private chain of thought. The constraint was a 3B local model: prompt-only control produced repeated searches, malformed arguments, copied examples and unfinished runs. I therefore made the model propose either a final answer or one structured tool call, while a deterministic controller owns validation, state and termination.

Scope ambiguity was addressed first. The UI had both a global scope toggle and note attachments. I replaced them with a normalised `linkedNoteIds` array, limited to eight. When notes are linked, the orchestrator reads them before the first model call, shares an 8,000-character context budget and removes whole-library `search_notes`. Without attachments, whole-library search remains available. The visible interaction is therefore enforced in code.

The original retrieval cascaded from keywords to embeddings: one literal match prevented semantic search. I run both routes in parallel. BGE-M3 was chosen for multilingual, multi-granularity retrieval (Chen et al., 2024), but keyword rank and cosine similarity are different scales. Results are therefore fused using Reciprocal Rank Fusion (Cormack et al., 2009):

$$\mathrm{RRF}(d) = \sum_{r \in \{\text{keyword,semantic}\}} \frac{1}{60 + \mathrm{rank}_r(d)}. \tag{0.1}$$

At most eight results are returned and labelled keyword, semantic or both. Semantic failure degrades to keywords with an explicit status instead of failing the Agent. Embeddings cover user-visible note content and are cached by content hash. This adapts the RAG principle of explicit non-parametric memory (Lewis et al., 2020), while keeping retrieval and evidence local.

3.2. Bounded model-tool loop

The controller registers `search_notes`, `read_note` and `extract_todos`. Each result returns as a `tool` observation for the next model decision, while the UI displays its query, source and retrieval route. Six code-level controls make the loop converge:

1. a six-step budget plus dynamic step/remaining state;
2. a registered-tool allow-list and argument validation;
3. note-scope checks before Repository reads or writes;
4. canonical call signatures that short-circuit duplicates;
5. `AbortSignal` cancellation around long-running tools; and
6. removal of all tools on the last step, forcing an evidence-based answer.

The complete action–observation path and the controller-owned invariants are shown in Figure B.2 in Appendix B.

The system prompt is organised as L0–L5 policy layers: role; facts/privacy; tool protocol; loop strategy; output format; and request-specific context. This is distinct from message roles such as system, user, assistant and tool. Lower-numbered rules have greater authority, so request context cannot override privacy. Prompt layers express intent; TypeScript guardrails enforce it.

Renderer code never imports models or the database. Calls pass through typed preload APIs and validated IPC to main-process services/Repositories; Agent turns and linked sources use the shared conversation store. Logs retain identifiers, sizes and error classes rather than raw private prompts. I rejected prompt-only looping because it failed to converge, keyword-first fallback because it hid semantic evidence, and a global scope toggle because it conflicted with attachments.

My supporting model work unified download, verification, activation, switching and runtime release for STT, LLM, embeddings and TTS. For TTS I integrated Kokoro, MeloTTS and MOSS behind one audio contract and selected MeloTTS as the balanced recommendation after comparison. Appendix B contains the shared architecture and model detail without claiming that I alone designed the architecture.

4. Evaluation Skills

I evaluated during development because a passing build cannot show that a 3B model uses evidence correctly, while a successful demo cannot prove scope, cancellation or persistence. Formative failures were retained to compare interventions; summative runs used fixed corpora, explicit denominators and generated result files. The final evaluation separates retrieval quality, model orchestration and runtime cost so that one subsystem cannot conceal another's weakness.

4.1. Retrieval quality versus Agent use

The retrieval corpus contained 80 notes and 24 labelled tasks. Production mixed retrieval ran keyword search and BGE-M3 in parallel, then fused their ranks with $k = 60$. When each task

instruction was sent directly to this pipeline, Recall@8 was 98.6%, MRR 0.939 and nDCG@8 0.952. The end-to-end Agent, however, found relevant evidence for only 41.7% of tasks at every cutoff (Figure C.2).

This isolates the main bottleneck: ranking is capable when given a useful query, but the model often declines to search, rewrites the query poorly or fails to read a result. Only 8.3% of tasks received a successful note-read action. The appropriate next change is therefore not another embedding model; it is better tool-selection training, query generation and search-to-read policy. The corpus is small and its instructions are more regular than natural user requests, so the direct result is an upper bound rather than a universal retrieval claim.

4.2. End-to-end Agent generalisation

The Agent corpus comprised 90 tasks split once into 45 development and 45 holdout items, executed with Qwen2.5-3B-Instruct and the production tool controller. Strict completion required correct tool behaviour, answer mode and task-specific outcomes. Holdout strict completion fell from 60.0% to 40.0%, while fact coverage increased to 94.8% (Figure C.1). The system usually possessed the necessary facts but did not consistently turn them into the required action or answer format. Answer-mode accuracy fell by 20 points, supporting this interpretation.

All successful scope-violation attempts were blocked, and duplicate-call, step-limit and cancellation probes passed. These code-owned safety properties are stronger than prompt compliance. Conversely, unanswerable and ambiguous holdout tasks achieved only 8.3% and 18.2% strict completion: abstention and clarification remain weak. An LLM judge passed 42.2% of holdout answers, but was not calibrated against humans, so I retain rule-based completion as the primary measure.

4.3. Model and runtime trade-offs

A five-model sweep showed that task extraction and Agent orchestration are distinct capabilities. Ministral-3-3B and Granite-4-Micro-H both reached 62.2% Agent completion, whereas the chosen Qwen2.5-3B baseline reached 48.9%. Qwen nevertheless had zero false positives on zero-task cases and 74.4 tokens/s. The faster Qwen2.5-1.5B reached 131.6 tokens/s but a 72.7% zero-task false-positive rate under the common prompt. The ranking also changed by responsibility: high single-step extraction F1 did not guarantee reliable multi-step tool use (Figure C.4). Therefore model choice must be task-specific rather than based on one aggregate capability score. Appendix C reports the full comparison and the asymmetric false-positive risk. A separately optimised, labelled task-extraction run used Qwen2.5-1.5B with a JSON-only prompt and deterministic gate, reaching 90.24% holdout F1 and 9.09% overall zero-task false positives. I do not combine these two configurations.

The task-specific ranking is shown in Figure C.4 in Appendix C, alongside the full numeric comparison.

The Windows TTS benchmark used 36 Chinese, English and mixed texts, three repetitions each, on an i9-12900H CPU with four threads. MOSS had the lowest median RTF (0.555) and lowest ASR-proxy CER (9.7%), but transient memory increased with text length and reached 10.16 GiB on the 1,196-character sample (Figure C.6). Kokoro and MeloTTS remained below 1 GiB on that sample; MeloTTS also had the smallest model asset. This supports MeloTTS as the balanced default and MOSS as an experimental option, not "fastest therefore best." Repeated calls showed no leak, so MOSS's problem is per-request allocation rather than cumulative retention.

STT evaluation used 56 recordings from one Chinese-native speaker. Whisper Small gave the best observed trade-off (17.0% CER at 0.45 RTF); Large-v1 was slower (2.62 RTF) without improving CER (18.0%). This does not establish population-level accuracy because speaker, microphone and accent diversity were absent.

Finally, the machine-readable inventory recorded 76 suites: 73 passed, none failed and three were skipped; 560 tests passed and 74 were skipped (Figure C.11). A four-machine run then exposed hardware-sensitive deployment behaviour: Qwen2.5-3B ranged from 43.9 to 233.5 tokens/s, while MOSS TTS peak RSS ranged from 5.5 to 18.6 GiB. These results strengthen feasibility and hardware-routing decisions, not quality claims, because software versions were not fully aligned. No participant usability study, multi-speaker STT study or blinded TTS listening test was completed.

5. Project Management Skills

I managed my contribution as risk-bearing vertical slices rather than separate frontend, AI and test phases. A model was complete only when it could be downloaded, verified, activated, exercised through packaged Electron, release its old runtime and pass relevant gates. An Agent change included UI, preload, IPC, service/Repository, cancellation and focused tests.

Thirteen dated FAN changelogs (12–23 August) recorded the problem, previous and new behaviour, files, verification and remaining risk. They acted as a work breakdown, decision log and handover. I prioritised blockers, privacy and data-loss risks before visual polish; reproduced external bug reports before changing code; and ordered work from model/process boundaries to Agent/retrieval, Conversation Studio, evaluation and packaging. Within each cycle I moved from the smallest failing case through focused tests, type-check, lint, build and visual review. Table A.1 records the four risk-led cycles without interrupting the ten-page body.

I kept group boundaries explicit. Models, Agent/retrieval and UI/HCI were my primary areas; architecture and database design were shared. Cross-cutting work followed the agreed Service/Repository to IPC to preload to Renderer route. Reproducible benchmark, smoke and evaluation scripts allowed others to compare later models. I also recorded skips, unsigned builds and untested platforms instead of turning them into successes. The same principle later enabled four contributors to run a machine-labelled hardware benchmark: I aggregated only matching mod-

els and workloads, retained missing values, and separated runtime measurements from machine-independent accuracy results.

The weakness of this method was that logs showed sequencing but not forecast accuracy. I did not maintain formal effort estimates or a burndown. Next time I would add weekly risk review, time budgets for evaluation and an earlier user-testing milestone while retaining the evidence-oriented definition of done.

Scope control was equally important. I deferred approximate-nearest-neighbour search, formal listening tests and unsupported platform claims when their prerequisites were absent, rather than letting them displace the core local workflow. When shared architecture or database changes were necessary, I raised the required interface and accepted the team-owned decision instead of redesigning adjacent systems. This kept my workload focused while still allowing my modules to integrate with group work.

6. Critical Self-Evaluation and Future Work

My strongest contribution was turning uncertain model behaviour into inspectable product behaviour. Model lifecycles made alternatives comparable; hybrid retrieval gave literal and semantic evidence a route into results; the controller bounded Agent actions; and the UI exposed context, readiness and progress. Recording failed attempts and platform limits also made the work more reproducible.

I would change three things. First, I trusted prompt wording for too long. The 9/17 checklist regression and early repeated Agent calls showed that deterministic state and evaluation should precede prompt polishing. Second, I treated some UI choices as styling until real rendering exposed performance and layout failures; frame-time and narrow-window budgets should have been defined earlier. Third, I did not arrange a formal usability study, so learnability, trust and disabled-user experience remain unknown.

A shared data migration also taught me to inspect dependencies outside my ownership. A

historical `userData` move could skip content when an apparently empty destination database already existed. My retrieval and conversation features depend on that data, so content-level migration should have been an acceptance condition. Likewise, runtime compatibility documents were not evidence of Windows/macOS performance. I initially reported this limit, then coordinated the same scripted workload across four machines. Because system and runtime versions were not fully aligned and most hosts ran once, the results demonstrate feasibility and variance, not hardware causality.

Future work should follow the remaining evidence risk. First, conduct task-based sessions with students and knowledge workers, including keyboard-only use, and add blinded TTS listening ratings. Second, repeat STT with multiple speakers, microphones and accents, then repeat the cross-machine benchmark with aligned versions, cold/hot runs and power settings. Third, improve search-query generation, search-to-read policy, abstention and clarification, then re-run the untouched Agent holdout set and calibrate the LLM judge against human ratings. The model sweep should also extend to 7B models and the VRAM boundary. Finally, add migration dry-runs, content checks and recoverable merging before scaling the vector index.

If repeating the project, I would establish evaluation datasets and user checkpoints with the first milestones. My central learning is that a useful local-AI product is defined not only by model capability, but by deterministic boundaries, recoverable data, comparative evidence and interactions that users can understand and reverse.

References

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-ai interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1–13. https://doi.org/10.1145/3290605.3300233

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). BGE M3-Embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*. https://doi.org/10.48550/arXiv.2402.03216

Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 758–759. https://doi.org/10.1145/1571941.1572114

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 154–178. https://doi.org/10.1145/3359591.3359737

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474.

References

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *Proceedings of the 40th International Conference on Machine Learning*, *202*, 28492–28518. https://proceedings.mlr.press/v202/radford23a.html

Shneiderman, B. (1983). Direct manipulation: A step beyond programming languages. *Computer*, *16*(8), 57–69. https://doi.org/10.1109/MC.1983.1654471

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629

A. Evidence Index

The following repository materials support the individual claims in this report. Files ending in FAN are my dated working records; source, tests and shared project documentation were used to verify final behaviour and contribution boundaries.

Table A.1.: Risk-led sequencing of my individual work

Cycle	Primary deliverable	Risk closed before moving on
12–13 Aug	Conversation Studio, model management, hybrid retrieval and TTS comparison	fragmented workflow, hidden runtime state and unmeasured model choice
16–18 Aug	regression repair, Agent scope, cancellation and persistence	cross-workspace access, lost conversations and silent readiness failure
20–21 Aug	real-model evaluation, bounded loop and direct manipulation	copied examples, private logs, repeated calls and inaccessible controls
23 Aug	unified note pipeline, responsive integration, packaging and documentation	duplicate knowledge output, narrow-layout regressions and unverifiable handover

A. Evidence Index

1. `docs/changelog/2026081201-FAN.txt`: Conversation Studio, language and initial Agent/-workspace integration.
2. `docs/changelog/2026081202-FAN.txt`: immediate recording-review feedback and four-part model management.
3. `docs/changelog/2026081301-FAN.txt`: attached-note context and RRF hybrid retrieval.
4. `docs/changelog/2026081302-FAN.txt`: hardware detection, UI corrections and runtime feedback.
5. `docs/changelog/2026081303-FAN.txt`: model presentation, workspace interaction and responsive fixes.
6. `docs/changelog/2026081304-FAN.txt`: three TTS engines, benchmark, packaging and platform limits.
7. `docs/changelog/2026081601-FAN.txt`: workspace-detail regression and page-level tests.
8. `docs/changelog/2026081801-FAN.txt`: Agent scope, task tool, cancellation, readiness and conversation persistence.
9. `docs/changelog/2026081802-FAN.txt`: design tokens, animation performance and spotlight onboarding.
10. `docs/changelog/2026081803-FAN.txt`: quality-gate repairs, packaging and unresolved migration/signing risks.
11. `docs/changelog/2026082001-FAN.txt`: real-model task evaluation, privacy logs and bounded prompt/controller design.
12. `docs/changelog/2026082101-FAN.txt`: direct manipulation, keyboard access, calendar linking and tutorial expansion.
13. `docs/changelog/2026082301-FAN.txt`: single structured-note pipeline, unified index, final Agent context and project gate.
14. `docs/testing/retrieval-eval.md` and `docs/testing/agent-end-to-end-eval.md`: fixed-corpus retrieval and Agent results, configurations and limitations.

15. `docs/testing/llm-model-sweep.md` and `docs/testing/task-extraction-eval.md`: common-baseline model comparison and separately labelled task-tool evaluation.
16. `docs/testing/stt-human-eval.md` and `docs/testing/tts-model-benchmark-windows.md`: human-recorded STT and full-corpus Windows TTS measurements.
17. `docs/testing/cross-machine-benchmark.md` and `docs/testing/results/machines/`: matching TTS, LLM and STT runtime measurements from four machines.
18. `docs/testing/m2-pro-16gb-hardware-benchmark-conclusion.md`: interpretation and limitations of the Apple M2 Pro replication.
19. `docs/testing/jest-test-inventory.md`: generated suite/test inventory and pass/skip counts.
20. `README.md`, `README.zh-CN.md` and `docs/project-structure.md`: shared final architecture, data model and quality snapshot.
21. `src/main/agent/AgentPrompt.ts`, `AgentOrchestrator.ts` and `AgentSearchNotesTool.ts`: prompt layers, bounded loop and RRF implementation.
22. `src/main/agent/__tests__/`: duplicate-call, scope, cancellation, final-step and prompt-policy tests.

B. Technical Figures and Model Detail

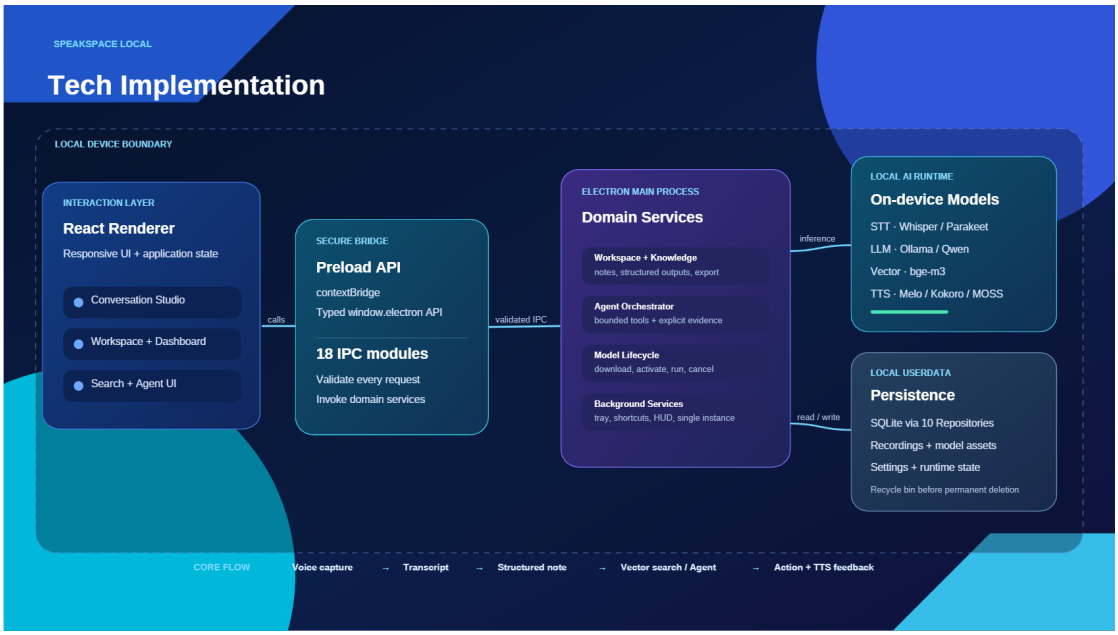

Figure B.1.: SpeakSpace Local implementation boundaries. My Agent and model-lifecycle work connects through typed preload/IPC; the wider architecture was collaborative.

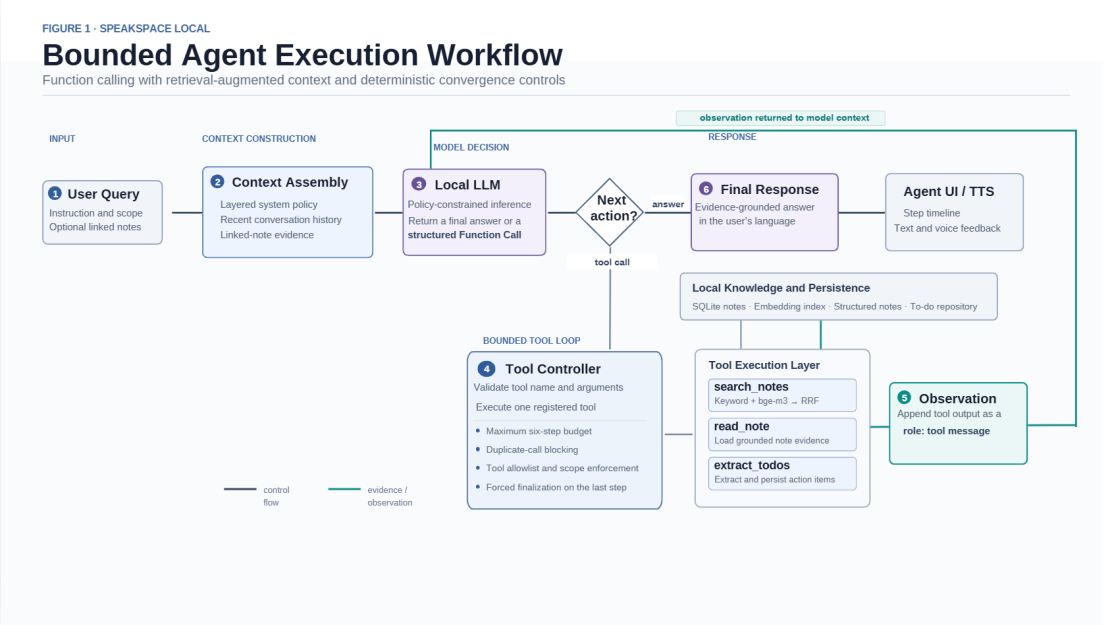

Figure B.2.: Bounded Agent execution: the local LLM selects an answer or registered tool, while controller invariants enforce scope, cancellation, duplicate-call and six-step bounds.

Table B.1.: Local model capabilities and integration responsibilities

Capability	**Runtime**	**Selection**	**Integration emphasis**
STT	Whisper CLI / Parakeet ONNX	Multilingual Whisper sizes plus fast English Parakeet	managed model location, activation and runtime readiness
LLM	local Ollama	Qwen2.5-3B-Instruct as the measured local baseline	shared service for questions, notes, tasks and Agent calls
Embedding	Ollama + BGE-M3	multilingual vectors without a second model server	batch embedding, validity checks, content-hash cache and fallback
TTS	sherpa ONNX / ONNX Runtime	Kokoro baseline, MeloTTS balanced choice, MOSS experimental	verified downloads, engine adapters, voices, channels and runtime release

C. Evaluation Detail

C.1. Agent failure profile

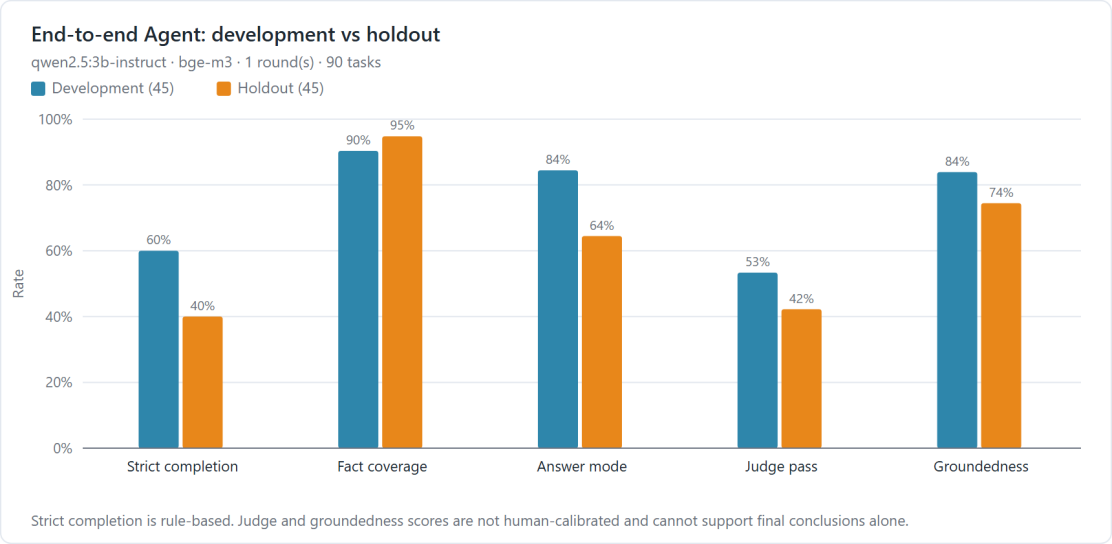

Figure C.1.: End-to-end Agent performance on the fixed 45-task development and 45-task hold-out split. The Judge and groundedness columns are shown for transparency but were not human-calibrated.

C. Evaluation Detail

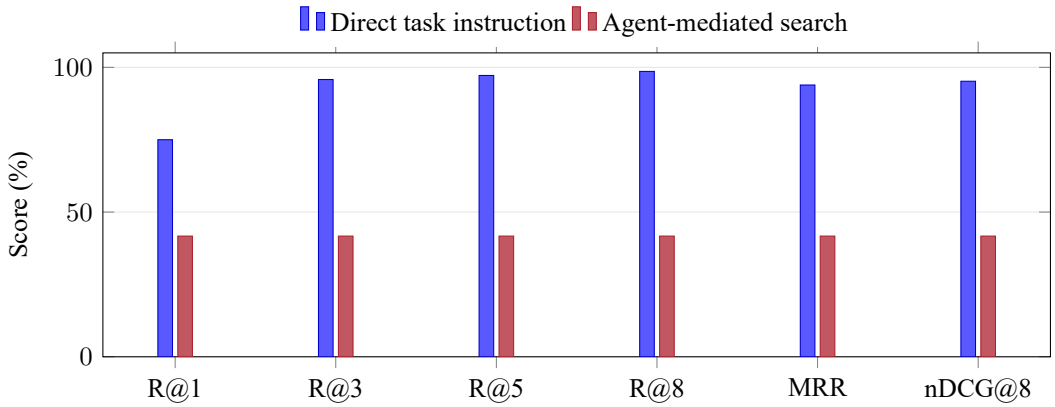

Figure C.2.: Direct retrieval is strong, but Agent-generated search and tool use create a 56.9-point Recall@8 gap. The comparison uses the 24 search-labelled tasks; it diagnoses orchestration rather than a change of index.

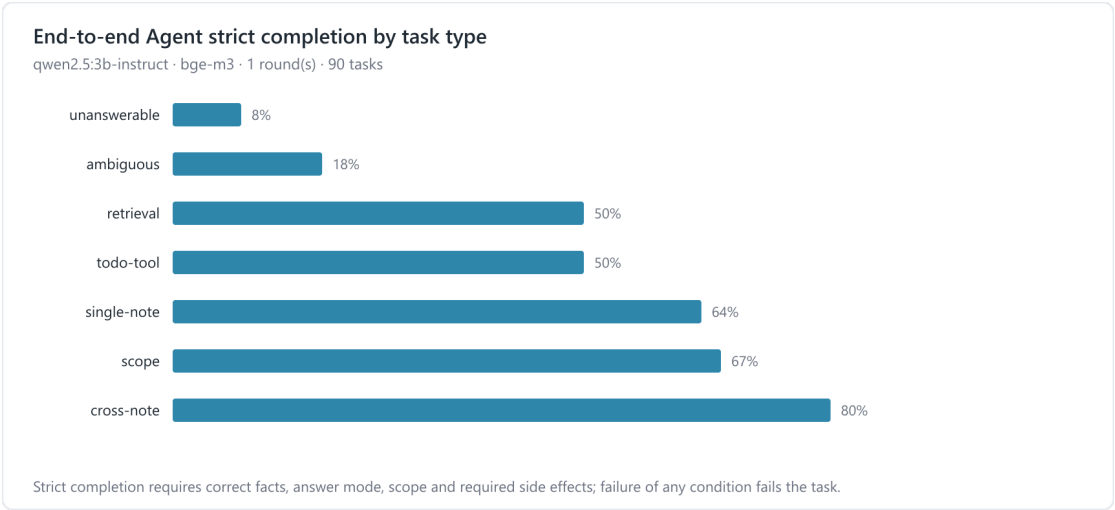

Figure C.3.: Strict completion across the seven Agent task types (90 fixed tasks, one run). The aggregate score conceals strong cross-note performance and weak abstention or clarification.

The 50.0% aggregate completion rate is therefore not a uniform capability score. Cross-note tasks reached 80.0%, whereas unanswerable and ambiguous tasks reached 8.3% and 18.2%. Because holdout fact coverage was already 94.8%, the next intervention should target answer routing, abstention and clarification rather than adding more retrieved context.

C.2. Model comparison and asymmetric errors

Table C.1.: Common-baseline LLM sweep on an RTX 3060 Laptop GPU

Model	**tokens/s**	**Todo F1**	**Zero-task FP**	**Agent completion**
Qwen2.5 3B	74.4	86.1%	0.0%	48.9%
Qwen2.5 1.5B	131.6	88.7%	72.7%	32.2%
Phi-4 Mini	57.1	63.4%	9.1%	36.7%
Ministral-3 3B	60.8	95.7%	6.1%	62.2%
Granite-4 Micro-H	56.8	94.2%	54.5%	62.2%

C. *Evaluation Detail*

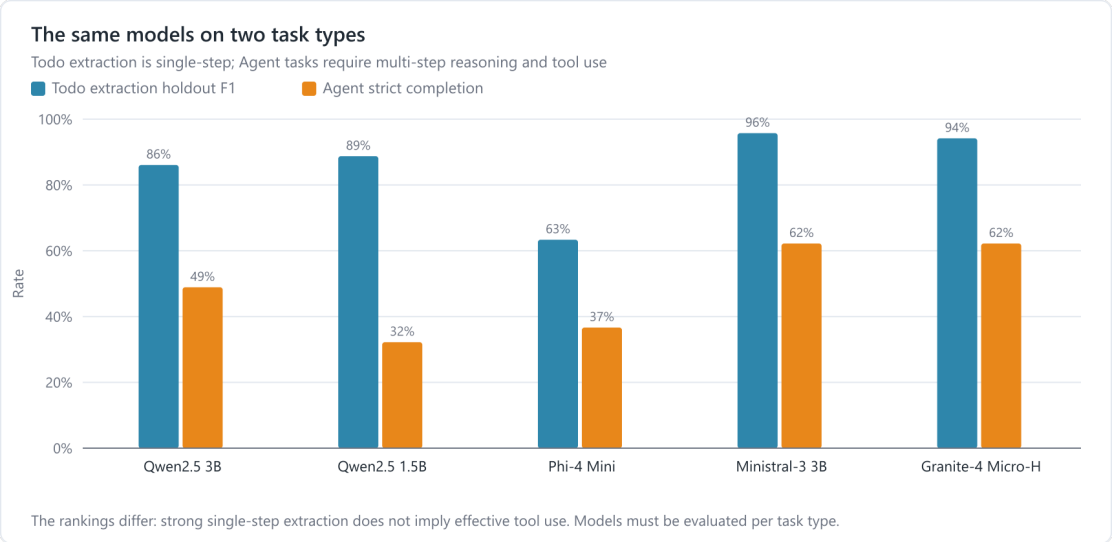

Figure C.4.: The common-baseline sweep ranks the same five models differently for Todo extraction and Agent orchestration. Todo F1 uses the 32-item holdout set; Agent completion uses all 90 tasks in one run.

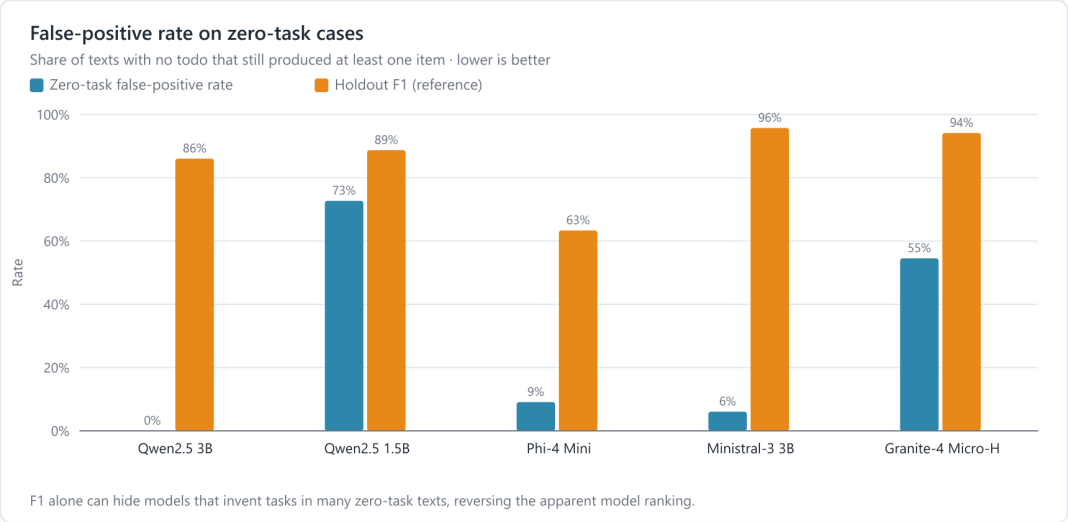

Figure C.5.: Todo holdout F1 conceals zero-task false positives. F1 uses 32 holdout cases; the false-positive rate averages three runs of the 11 zero-task cases.

Invented tasks change the user's list, so false positives cost more than misses. Qwen2.5-3B's 0.0% rate supports a baseline; Granite-4's 54.5% requires a gate. The optimised 1.5B run uses a different prompt and gate.

C.3. Speech model trade-offs

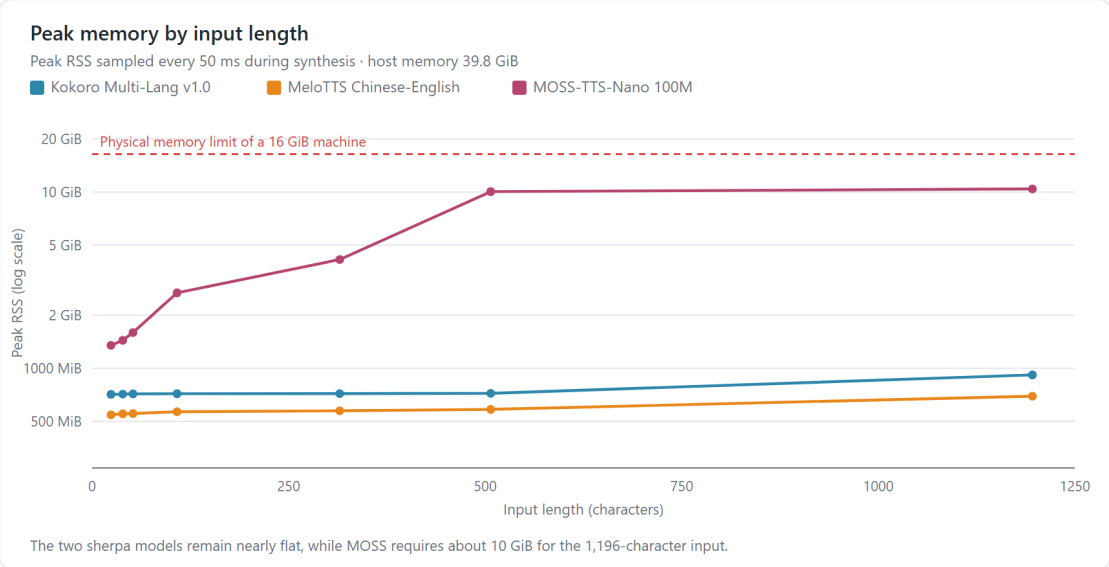

Figure C.6.: Peak resident memory against input length on the Windows benchmark host. MOSS allocation grows with text length while Kokoro and MeloTTS remain below 1 GiB on the 1,196-character probe.

Table C.2.: TTS comparison on Windows x64, i9-12900H CPU, four threads, 36 texts with three repetitions

Model	P50 RTF	Corpus peak RSS	1,196-char RSS	ASR-proxy CER
Kokoro Multi-Lang	0.825	1,381.8 MiB	918.9 MiB	10.3%
MeloTTS zh-en	0.619	751.7 MiB	698.4 MiB	17.1%
MOSS-TTS-Nano	0.555	19,033.1 MiB	10,407.2 MiB	9.7%

RTF below one is faster than playback. All three engines met that threshold, so the large separation in peak memory is more decision-relevant than the smaller speed difference. ASR back-transcription remains an intelligibility proxy, not a human listening-quality score.

Table C.3.: Human-recorded STT comparison (56 recordings, one Chinese-native speaker)

Whisper model	**A+C CER**	**RTF**
Tiny	36.5%	0.09
Base	27.1%	0.14
Small	17.0%	0.45
Large-v1	18.0%	2.62

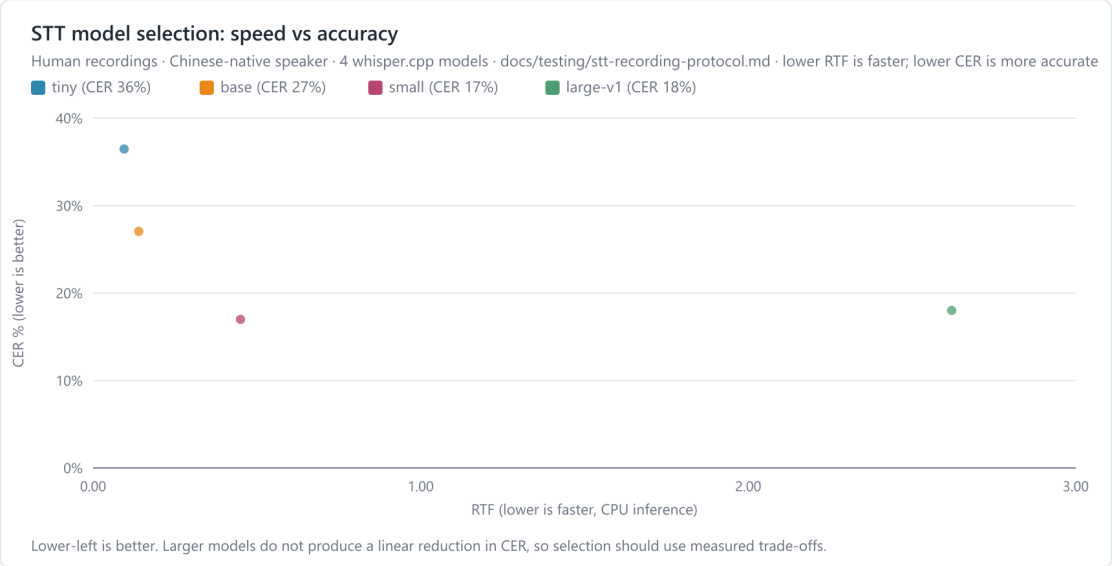

Figure C.7.: Observed STT speed–accuracy trade-off on 56 human recordings. Whisper Small occupied the best measured compromise; Large-v1 was slower without reducing CER.

This result supports Whisper Small for the tested device, but not a population-level claim: the corpus contains one speaker and does not vary accent, microphone or room noise.

C.4. Cross-machine feasibility

Four contributors ran the same hardware-sensitive workload on three Windows/NVIDIA hosts and one Apple M2 Pro host. Only matching model–workload pairs were aggregated; missing results were not treated as zero. The measurements establish feasibility and host sensitivity, not a causal hardware ranking, because system and runtime versions were not fully aligned and most hosts completed one full run.

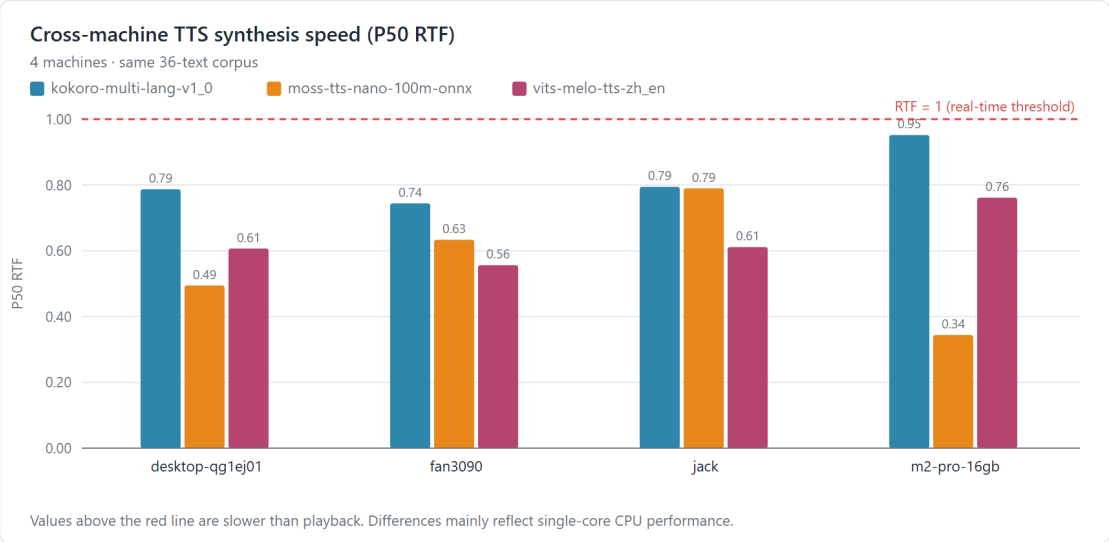

Figure C.8.: Median TTS real-time factor on four machines using the same 36-text corpus. Every model–host pair remained below the real-time threshold, although the relative ranking changed.

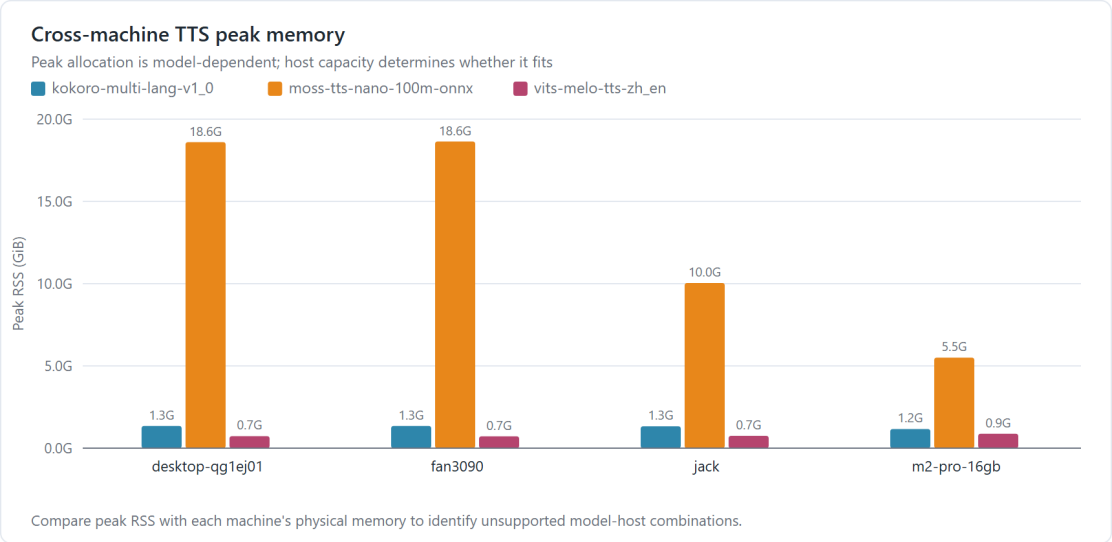

Figure C.9.: Peak process RSS for the same TTS workload. MeloTTS stayed within 0.7–0.9 GiB, whereas MOSS ranged from 5.5 to 18.6 GiB, reinforcing a hardware-aware experimental label.

C. *Evaluation Detail*

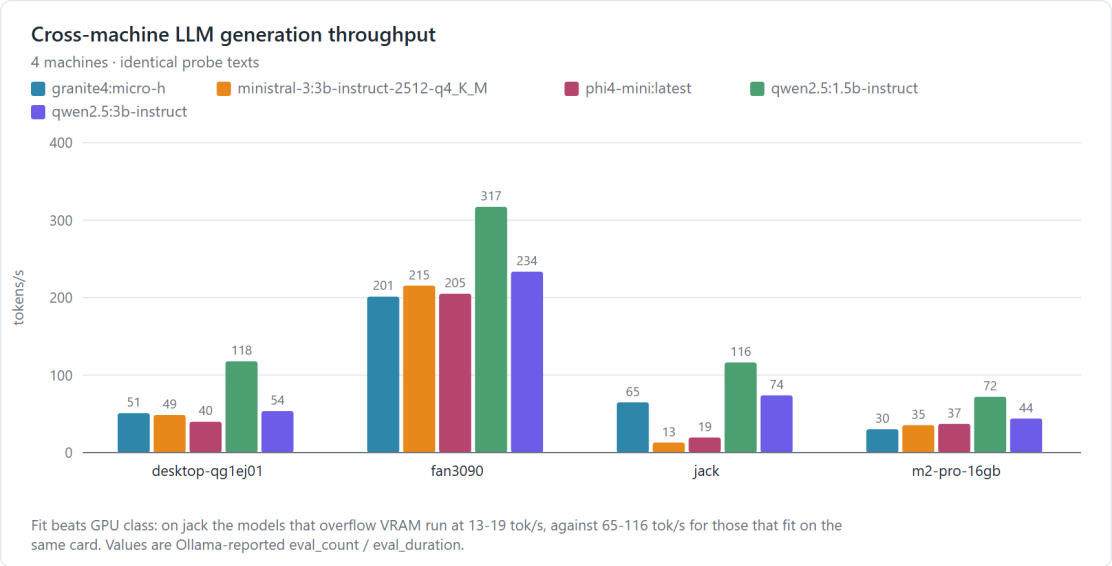

Figure C.10.: Ollama-reported generation throughput for five local LLMs. Qwen2.5-3B ranged from 43.9 tokens/s on M2 Pro to 233.5 tokens/s on the RTX 3090 host; throughput therefore cannot be presented without hardware context.

The RTX 3050 host only offloaded 52% of Ministral-3 and 64% of Phi-4 Mini, coinciding with their lowest observed throughput. All other recorded model–host pairs reported full offload. For STT, the M2 Pro was between 2.00 and 8.51 times faster than the RTX 3060 laptop on the 13 Whisper variants shared by both results; this is runtime evidence only, because accuracy for identical recordings is machine-independent.

C.5. Automated regression evidence

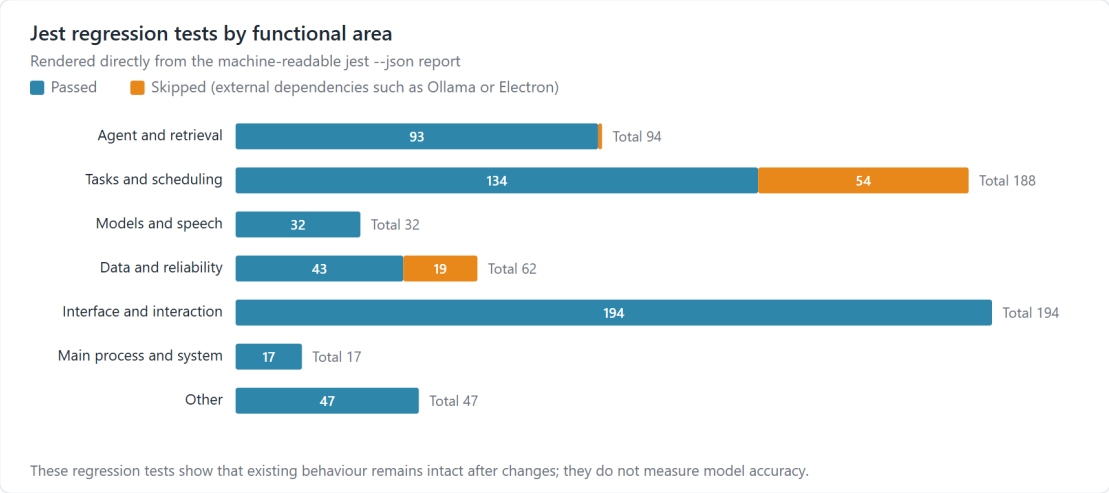

Figure C.11.: The generated Jest inventory groups all 634 tests by functional area: 560 passed, none failed and 74 requiring external conditions were skipped. These are regression checks, not model-accuracy measurements.

The 76 suites included 94 Agent/retrieval tests, 188 task/scheduling tests, 32 model/speech tests, 62 data/reliability tests, 194 interface/interaction tests and 64 other main-process or supporting checks. Reporting the categories makes the project gate auditable without presenting 560 assertions as independent user scenarios.

C.6. Evaluation-to-decision traceability

Table C.4.: How evaluation findings affected implementation

Observation	Action	Residual limitation
Agent retrieval lagged direct retrieval by 56.9 Recall@8 points	prioritised query/tool policy over replacing BGE-M3	24 regular task instructions are not natural-query coverage
Strong final negative checklist reduced 15/17 to 9/17	reverted the intervention and retained the failure in the log	small models remain sensitive to prompt position
MOSS was fastest but reached 10.4 GiB on the longest text	recommended MeloTTS; kept MOSS experimental and Kokoro as baseline	ASR CER is not a listening-quality score
Page-transition blur stuttered in a model-heavy view	removed frame-by-frame blur; kept transform and opacity	no automated frame-time threshold yet
Linked notes could be ignored by Agent search	preloaded linked content and removed whole-library search	eight-note and 8,000-character limits require future user evaluation